

计算机安全专题报告：恶意软件分析综述

第二组：盖乐 陈昕怡 王迪 贾泽栋 陈奕昊

摘要

随着移动互联网与物联网技术的飞速发展，恶意软件的变异速度与攻击手段的隐蔽性呈指数级增长。传统的基于静态特征匹配或简单统计学规律的检测方案，在面对代码混淆、加壳技术以及 API 快速迭代产生的“概念漂移”问题时，逐渐显露出鲁棒性不足与维护成本高昂的缺陷。本报告基于计算机安全汇报的第一部分核心内容，对四篇具有里程碑意义的学术论文（MAMADROID, HinDroid, APIGRAPH, DawnGNN）进行了深度剖析。报告首先详细阐述了各研究在行为建模、异质网络挖掘及语义知识融合方面的创新机制；随后，通过系统化知识建模（SoK）的方法，从横向的技术范式对比与纵向的时间演进脉络两个维度，对恶意软件检测领域的技术图谱进行了重构与展望。

一、Level 1：传统恶意软件检测技术

（一）MAMADROID：基于行为模型马尔可夫链的安卓恶意软件检测

本文的研究动机源于安卓生态系统的快速演演进。传统的特征提取方法，如单纯依赖 API 调用频率，往往难以捕捉程序运行的时序逻辑，且随着安卓 API 的不断更新，模型的鲁棒性会迅速下降。该研究旨在解决恶意软件检测中特征维度过高、稀疏性严重以及对系统更新过于敏感的挑战。

为了应对这些挑战，MAMADROID 提出了一种基于抽象 API 调用的行为建模方法。其核心思路是将具体的 API 调用序列通过两个层级的抽象（包级别 Package 和家族级别 Family）进行泛化。例如，将具体的函数调用映射到所属的 Java 包，从而降低了特征空间的复杂度。在此基础上，研究者引入了马尔可夫链模型，将抽象后的包或家族视为状态，计算不同状态之间的转移概率，从而构建

出反映程序执行逻辑的转移概率矩阵。通过这种方式，原本离散的 API 序列被转化为具有统计学意义的特征向量。

MAMADROID 在涵盖六年跨度的 8.5K 个良性应用和 35.5K 个恶意样本的数据集上进行了严格评估，其 F-measure 达到了惊人的 99%，证明了行为时序特征的高区分度。更为引人注目的是其抗老化能力：实验显示，使用两年前的数据训练的模型，在检测当年的新样本时，性能衰减幅度显著小于传统的基于 API 频率的方法（如 DroidAPIMiner）。

然而，该方案并非完美无缺。由于采用了高度抽象的方法，模型不可避免地丢失了某些细粒度的语义信息（如具体的函数参数值）。这导致在面对经过精心构造、且在宏观逻辑上符合正常应用规范的攻击时，模型可能出现误报或漏报。未来的改进方向应聚焦于如何引入更复杂的语义分析以平衡抽象程度与检测精度，或者结合动态执行数据来增强对隐蔽行为的捕捉能力。

（二）HinDroid: 基于结构化异质信息网络的智能安卓恶意软件检测系统

随着对抗技术的升级，简单的代码特征已不足以应对复杂的恶意软件。攻击者可以通过各种混淆技术（如重命名、控制流扁平化）或动态加载技术来隐藏其恶意的 API 调用，从而规避基于单一特征的检测。研究动机在于，简单的 API 调用分析无法反映应用、API 及其复杂环境之间的内在联系。因此需要一种更高层次的语义表示来增加攻击者的规避成本。

HinDroid 解决的核心挑战是如何在统一的框架下建模应用与 API 之间多维度、异构的关系。研究者提出将安卓应用（Apps）、相关的 API 及其关系构建为一个结构化的异质信息网络（HIN）。在这个网络中，不仅包含“应用调用 API”的关系，还引入了“API 属于同一包”、“API 在同一代码块中出现”等更高阶的关系。通过预定义的一系列元路径（Meta-paths），如“App-API-App”或“App-API-Package-API-App”，系统能够量化应用之间的语义相似性。随后，利

用多核学习（Multi-kernel learning）算法，将不同元路径生成的相似度矩阵进行加权融合，自动识别对检测最有贡献的关系组合。

实验结果表明，HinDroid 在处理大规模样本时具有极高的准确率和召回率。这种基于图结构的方法使得特征提取不再局限于孤立的属性，而是深入到了代码的结构化关联中。但其不足点在于，HIN 的构建和元路径的计算具有较高的计算复杂度，对于实时检测场景可能存在性能瓶颈。未来的改进方向在于优化大规模图数据的索引与查询效率，并探索如何动态更新 HIN 以适应快速生成的恶意样本。

（三）APIGRAPH: 利用 API 知识延缓学习型恶意软件检测器的老化

本文直接针对模型老化这一核心痛点。随着时间推移，恶意软件开发者会使用语义等效的新 API 替换旧 API 以逃避检测，或者操作系统本身会废弃旧 API。传统的解决方法是持续不断地对新样本进行人工标注并重新训练模型，这需要耗费巨大的人力成本和时间周期，且往往滞后于威胁的演变。

APIGRAPH 的设计思路是从特征空间入手，通过捕获 API 的本质语义来提高模型的稳定性。研究者观察到，虽然 API 的实现形式在变，但其在官方文档中的功能描述通常是稳定的。因此，他们开发了 APIGRAPH 系统，自动从 API 文档中提取知识，构建 API 关联图。在该图中，具有相似功能或在文档中共同出现的 API 被连接在一起。通过这种语义关联，系统可以将不同版本的 API 映射到同一个潜在的语义空间中，使得检测器能够识别出那些“换汤不换药”的恶意变体。

APIGRAPH 成功增强了 5 个最先进的恶意软件检测器，涵盖了安卓和 Windows 平台。实验证明，该方法能显著延缓模型随时间推移的性能衰减。不足之处在于，该系统高度依赖于 API 文档的质量和完整性。如果文档描述模糊或存在缺失，提取的语义关联可能存在噪声。未来的研究可以考虑融合多源信息（如 Stack Overflow 等开发者社区的讨论）来进一步丰富 API 的知识表示。

（四）DawnGNN：文档增强的 Windows 恶意软件检测图神经网络

本文延续了语义增强的思路，但将其应用到了 Windows 平台的动态 API 分析中。研究动机在于，传统的 Windows 恶意软件分析往往只关注 API 序列的统计特征，忽略了官方 API 文档中蕴含的丰富行为描述，这限制了模型对复杂攻击意图的理解能力。

DawnGNN 提出的具体思路是将自然语言处理技术（NLP）与图神经网络（GNN）深度融合。首先，利用预训练的 BERT 模型对 Windows 官方文档进行建模，将每个 API 的文字描述转化为高维的语义嵌入向量（Embedding）。接着，将应用运行时的 API 调用序列转化为 API 调用图，其中节点不仅包含 API 的标识，还携带了从文档中获得的语义特征。通过图同构网络（GIN）等 GNN 变体，系统能够同时学习调用图的拓扑结构和 API 的深度语义。

在最终效果上，DawnGNN 在多个公开数据集上超越了现有的动态分析方案，证明了外部知识库（文档）在提升恶意软件理解深度方面的巨大潜力。然而，作为一种基于动态分析的方法，它不可避免地受到环境模拟不完全的限制（如反沙箱技术导致恶意行为未触发）。此外，BERT 模型和 GNN 的推理成本相对较高。改进方向包括结合静态分析特征构建多模态模型以提升覆盖率，以及优化 GNN 架构以提升推理效率，使其更适合端侧部署。

技术路线与方法论的横向对比

● 恶意软件检测技术的演进路径（纵向对比）

从上述四项研究中，我们可以清晰地观察到恶意软件检测技术的纵向演进脉络，即从“结构抽象”到“关联建模”，再到“语义驱动”的转变。

早期的 MAMADROID 代表了结构抽象阶段，其核心是通过层次化的类别划分来对抗 API 的爆炸式增长和频繁更新。这一阶段解决了“特征稀疏性”问题，但特征的表达能力仍停留在统计概率层面。随后的 HinDroid 进入了关联建模阶段，

利用 HIN 和元路径技术，将检测维度从单一节点扩展到了复杂的关系网络，捕捉到了代码逻辑中的深层结构。

近期的 APIGRAPH 和 DawnGNN 则标志着语义驱动阶段的到来。研究者不再满足于代码本身的结构，而是引入了外部知识库（官方文档）。通过 BERT 等 NLP 模型，API 不再是冰冷的字符串或索引，而是具有特定功能含义的实体。这种从“形式”到“意图”的跨越，极大地提升了检测器对抗变体和模型老化的能力。

● 技术路线与方法论的横向对比

在横向对比中，四篇论文展示了不同的建模范式。MAMADROID 基于概率图模型（马尔可夫链），侧重于行为的时序性和概率转移，具有计算高效、逻辑直观的优点，但在处理长程依赖关系时略显不足。HinDroid 则基于图论中的异质网络，侧重于静态关系挖掘，其表达能力极强，但受限于元路径的手工设计成本和计算开销。

相比之下，APIGRAPH 和 DawnGNN 代表了现代深度学习与领域知识结合的趋势。APIGRAPH 更像是一个特征增强层，可以插件式地提升现有模型的效果；而 DawnGNN 则是端到端的架构，通过 GNN 直接在图结构上进行学习。两者都成功解决了“语义丢失”的问题，但在对动态行为的支持上，DawnGNN（动态 API 流）比 APIGRAPH（偏重静态映射）更具实时分析的潜力。

二、Level 2：表征驱动，鲁棒化恶意软件分类

在恶意软件检测领域，机器学习与深度学习的引入显著提升了自动化分析能力。随着 Level-1 中以控制流图（CFG）、API 调用图、行为序列等为代表的结构化表征逐步成熟，模型已经能够在统一的特征空间中学习恶意与良性程序之间的统计差异。然而，当这些模型从实验室评测走向真实部署环境时，其性能往往迅速退化，暴露出传统“高精度检测”范式的根本局限。

这一问题并非简单的模型欠拟合或特征不足，而是源于现实世界中数据生成机制的持续变化、不确定性以及对抗性威胁的加剧：一方面，恶意软件不断演化，通过加壳、混淆、重用代码模板等方式规避检测；另一方面，数据集本身存在不可忽视的噪声与偏差，例如标签来源不稳定、样本分布不均衡以及实验评估设置与真实部署环境不匹配等；此外，攻击者主动利用机器学习模型的脆弱性，通过对抗样本或后门投毒等手段操纵模型决策，使得检测器在面对精心设计的攻击时完全失效。由此，研究重心逐渐从如何提高单次训练下的分类准确率，转向了如何构建在长期运行中仍具备稳定判别能力且对对抗性威胁具有抵抗力的可信检测系统。

相较于 Level-1 关注"如何为模型提供可学习的表征"，Level-2 更关注判别式模型在复杂、动态、对抗性环境中的鲁棒性与泛化能力。其核心问题可以概括为五类：

- （1）评估与数据构造偏差导致的虚高性能；
- （2）大规模数据集中普遍存在的标签噪声；
- （3）随时间演化产生的概念漂移；
- （4）对抗样本攻击对检测器的逃避威胁；
- （5）训练阶段的后门投毒攻击与防御。

本报告围绕上述问题，对八篇代表性工作进行系统梳理与比较，展示判别式恶意软件检测在应对现实挑战时的技术演进路径——从认知评估偏差的根源，到修正标签噪声、应对概念漂移，再到理解并防御对抗性威胁。

（一）Decoding the Secrets of ML in Malware Classification（CCS 2023）

1. 研究动机

近年来，恶意软件检测领域大量研究报告了接近完美的检测性能（如 $F1 \approx 0.99$ ），给人一种“该问题已被解决”的印象。然而，这些结果往往建立在不同数

数据集、不同特征工程策略、不同家族定义与评估设置之上，使得研究结论缺乏可比性。作者认为，社区尚未真正理解：模型性能究竟是来源于方法本身，还是源于数据集构造与实验假设。

因此，该工作的动机并非提出新的检测模型，而是系统性地解构现有恶意软件分类结果的来源，澄清影响判别式模型性能的关键因素。

2. 研究挑战

该研究面临的核心挑战并非技术实现本身，而在于实验控制与公平比较。判别式恶意软件检测领域中大量看似优异的结果，往往建立在一组隐含且未被统一检验的研究假设之上。不同研究在数据集规模、家族数量及样本分布上的差异，使得任务难度并不一致，而家族分类作为多类别、长尾分布问题，对这些因素尤为敏感，进而削弱了实验结果的可比性。同时，静态特征、动态特征及其融合策略的有效性长期依赖经验性判断，缺乏在统一条件下的系统验证：动态分析受执行环境与路径覆盖限制，而静态特征虽更稳定却常被低估。此外，恶意软件家族标签本身并非稳定语义真值，其由多引擎共识生成且随时间演化，使标签噪声成为结构性问题。在未对上述假设进行显式控制的情况下，模型往往学习到的是数据构造与标签机制中的偏差，而非可在真实部署中长期成立的判别规律，这也直接促使后续研究转向标签噪声、概念漂移与不变特征等问题。

3. 解决挑战的具体思路

为系统性回应上述挑战，作者并未提出新的检测模型，而是采用大规模受控测量研究的方法，对现有判别式恶意软件检测结论进行系统剖析。该工作构建了当时规模最大且严格平衡的 Windows 恶意软件数据集（约 67,000 个样本、670 个家族），并在统一实验框架下对静态特征、动态特征及其融合策略进行对比评估，以消除不同实验设置带来的不可比性。同时，作者通过控制变量分析家族数量、单个家族样本规模及分布均衡性等因素对检测与家族分类性能的影响，从而揭示性能差异背后的结构性原因。通过同时考察检测与家族分类等多种任务，该研究显著提升了结论的稳健性与外推价值。

4. 最终效果

实验结果揭示了一系列对恶意软件检测研究社区具有“纠偏”意义的结论。在统一评测条件下，基于静态特征的方法整体性能优于动态特征，而静态与动态特征的融合仅带来有限的边际收益，挑战了“动态分析必然更优”的经验性认知。同时，研究表明恶意软件家族数量的增加会显著提升分类难度，而单个家族样本规模的扩大有助于提升分类精度，凸显了多类别、长尾分布特性对检测性能的影响。此外，在家族分布均衡的数据集上训练的模型表现出更强的泛化能力。针对动态特征，实验进一步指出运行时行为缺失会显著损害模型性能，反映出其对执行覆盖率和分析稳定性的高度依赖。

5. 不足与改进方向

该工作本身不直接解决概念漂移或标签噪声问题，其价值在于揭示问题而非修复问题。它假设研究者有能力获取高质量、受控的数据集，这在真实工业环境中并不总是成立。不过总整体来看，它也算是 Level2 研究中的基石，要求我们在讨论鲁棒性与泛化之前，必须首先澄清性能从何而来，该工作为后续关于标签噪声与概念漂移的研究提供了可信的评价基准。

（二）Differential Training: A Generic Framework to Reduce Label Noises for Android Malware Detection (NDSS 2021)

1. 研究动机

在大规模恶意软件检测中，训练数据通常依赖 VirusTotal 等多引擎扫描平台生成标签，而这些标签不仅在不同安全引擎之间存在显著分歧，还会随着时间推移发生变化。由于人工核验在规模上不可行，标签噪声成为恶意软件检测中一种结构性且长期存在的问题。Differential Training 的研究动机正源于此：在不存在干净标注集、且不引入人工干预的现实约束下，是否仍有可能自动识别并修正错误标签，从而提升判别式模型的可靠性。

2. 研究挑战

这一目标面临多重现实挑战。首先，在大规模恶意软件数据集中，方法无法合理假设存在一小部分可作为参照的“可信黄金样本”，因为标签通常来源于多引擎扫描结果，其一致性本身具有不确定性，且难以事后验证。其次，错误标签在真实数据中的比例往往未知，并且可能随时间与数据来源发生变化，这使得噪声既会干扰模型在训练阶段对判别边界的学习，也会污染评估阶段对模型性能的判断，从而形成反馈放大的误差。再次，已有的噪声鲁棒学习方法多依赖明确的先验假设（如噪声比例上界、噪声分布形式）或需要人工规则与干预，这类假设在恶意软件检测这一高度动态、异构且对抗性强的领域中往往难以成立，限制了相关方法在真实场景中的可用性。

3. 解决挑战的具体思路

为应对上述挑战，作者提出了利用训练动态作为噪声判别信号的 **Differential Training** 框架，其核心思想是在不依赖额外先验或人工标注的前提下，通过比较样本在不同训练条件下的学习行为来识别潜在的错误标签。具体而言，该方法同时训练两个结构相同的深度分类模型，一个使用完整训练集，另一个使用随机下采样的子集，由于训练数据规模与组成不同，样本在两个模型中的训练动态会呈现出差异化模式。

在训练过程中，**Differential Training** 并不局限于模型最终的预测结果，而是记录每个样本在整个训练阶段跨多个 `epoch` 的 `loss` 变化序列，并将同一样本在两个模型中的 `loss` 序列拼接为“噪声检测特征向量”，以刻画其训练行为的一致性。基于“正确标注样本在不同训练视角下表现更稳定，而错误标签样本更具波动性”的观察，作者进一步通过离群点检测识别训练动态显著异常的样本，并对其标签进行修正。该过程可迭代执行，从而逐步净化训练数据集并提升后续检测模型的鲁棒性。

4. 最终效果

实验结果表明，在 **Drebin**、**DeepRefiner** 等多种检测器以及不同规模的数据

集上，Differential Training 能够识别出大量错误标签；在完成重标注后，模型性能显著提升，其 F-score 接近使用全真标签训练时的性能上界，并明显优于 Co-Teaching 等主流噪声鲁棒方法。

5. 不足与改进方向

从 SoK 的视角来看，Differential Training 代表了 Level-2 中一条数据层鲁棒性的典型路径：它不试图改变模型结构或特征表达，而是通过理解模型训练行为来修复数据本身的问题。其隐含假设是世界是静态的，分布并未发生变化，唯一需要处理的是标签的不可靠性。这一假设在静态数据集场景下合理，但也构成了该方法在时间演化环境中的潜在局限。

（三）Continuous Learning for Android Malware Detection（USENIX 2023）

1. 研究动机

与关注静态数据可信性的研究不同，Continuous Learning 明确将研究对象置于长期真实部署环境之中，强调模型在时间维度上的持续有效性。作者通过真实时间跨度的实验观察发现，即使是在高质量历史数据上训练、在离线评测中表现优异的 Android 恶意软件检测器，其性能也会因恶意软件形态与良性应用行为的持续演化而迅速退化。这一现象表明，恶意软件检测并非一个静态判别问题，而是一个随时间不断变化的动态过程，一次性训练的判别模型难以在长期运行中保持稳定性能，概念漂移已成为影响实际部署效果的核心瓶颈。

2. 研究挑战

恶意软件与良性应用的功能与行为会随时间不断演化，直接导致数据分布发生漂移，使得历史数据中学习到的判别边界逐渐失效。其次，人工标注资源在现实安全场景中极为有限，无法支撑对新样本的持续、大规模人工分析。此外，真实应用生态中类别分布高度不平衡，良性样本占据绝对多数，使得传统基于预测

不确定性的样本选择策略在漂移场景下容易受到噪声干扰，难以稳定识别真正具有信息价值的样本。这些因素共同增加了持续学习在恶意软件检测中的复杂性。

3. 解决挑战的具体思路

作者提出了一种结合主动学习与对比学习的持续更新框架，其核心思想并非仅依赖模型对单个样本的预测置信度，而是通过对比学习构建更加稳定、结构化的嵌入空间，使模型能够基于样本之间的相对关系来感知分布变化。具体而言，作者设计了分层对比学习机制，在嵌入空间中显式强化同一恶意软件家族样本之间的相似性，同时仅要求不同家族恶意样本保持“弱相似”，从而在一定程度上缓解类别不平衡与家族差异带来的干扰。在此基础上，作者进一步提出 `pseudo loss` 作为对比学习场景下的不确定性度量，用以衡量样本在嵌入空间中的不稳定程度，并据此选择最具信息量的样本交由人工标注。模型则通过 `warm-start` 策略进行持续更新，避免频繁从头训练带来的性能波动与计算开销。

4. 最终效果

在跨越七年的真实 Android 恶意软件数据集上，该方法在多项指标上均显著优于多种基线方案。实验结果显示，在长期运行场景中，该方法将漏检率从 14% 降至 9%，同时有效控制了误报率的增长；更为重要的是，在达到相同性能目标的前提下，所需的人工标注样本数量显著减少。这表明，通过引入基于相似性建模的样本选择机制，持续学习框架能够在概念漂移环境下实现更稳定且更具成本效益的性能维护。

5. 不足与改进方向

从 SoK 的视角来看，该工作体现了 Level-2 中一种工程化、闭环式的鲁棒性范式。它并未回避分布变化这一现实前提，而是将概念漂移视为系统运行中的常态，通过人机协作不断修正模型的判别边界。然而，这一路径的代价也十分明确：模型性能的长期稳定性在很大程度上依赖于人工标注资源的持续投入，其可扩展性与适用范围受制于标注成本和人力资源的可持续性，这也为后续研究探索

更自动化的漂移应对机制留下了空间。

（四）Revisiting Concept Drift in Windows Malware Detection（NDSS 2025）

1. 研究动机

Revisiting Concept Drift 在前述持续学习范式的基础上，进一步对“主动学习 + 重训”这一主流策略提出了系统性的反思。作者指出，在现实部署环境中，漂移后可获得的新样本往往十分有限，在这种低样本条件下，简单地使用少量新数据对模型进行重训，往往并不能真正提升其对新分布的泛化能力。相反，模型可能继续依赖仅在漂移前存在、但在漂移后逐渐失效的“捷径特征”，从而在表面上完成更新，却未能在本质上适应分布变化。

2. 研究挑战

这一问题的核心挑战在于多个因素的共同作用。首先，漂移发生后可用的标注样本数量通常十分有限，难以支撑模型在新分布上进行充分学习；其次，深度模型具有强大的拟合能力，容易捕捉域特异的统计相关性，从而在训练过程中偏向于短期有效但不可持续的特征模式；与此同时，有效的恶意软件检测需要依赖在不同时间阶段均稳定存在的行为或结构特征，这对模型的表征学习能力提出了更高要求。

3. 解决挑战的具体思路

为应对上述挑战，作者提出从表征层面显式学习漂移不变特征（**drift-invariant features**），将研究重心从模型更新策略转向特征抽象机制。具体而言，该方法以 Windows 恶意软件的控制流图（CFG）作为基础表征，利用图神经网络对程序的结构与语义信息进行建模，并在此过程中引入对抗式域自适应机制。在训练阶段，分类器被优化以区分恶意与良性样本，而域判别器则试图区分样本来自漂移前还是漂移后分布；通过二者之间的最小最大博弈，模型被迫学

习在时间维度上不可区分、但在判别任务上仍具有有效性的中间表示，从而抑制对域特异特征的依赖。

4. 最终效果

在 2024 年真实恶意软件样本流以及多个公开基准数据集上，该方法在检测性能与鲁棒性方面均显著优于现有 SOTA 的主动学习与重训策略。更为重要的是，在达到相同检测性能的前提下，该方法所需的人工标注样本数量可降低约 5 倍，显示出良好的样本效率；同时，在多恶意软件家族的时间演化预测任务中，模型仍能保持稳定表现，验证了其对长期分布变化的适应能力。

5. 不足与改进方向

从系统化知识的角度来看，该工作标志着 Level-2 研究范式的一次重要升级，即研究重心从“如何更高效地适配漂移”转向“如何主动抽象并保留跨域不变性”。这一转变为构建长期可信的判别式恶意软件检测系统提供了更为根本的理论与方法方向，有助于减少对持续人工干预的依赖。然而，该方法对控制流图构建与图神经网络计算的依赖，也不可避免地带来了较高的工程与计算成本，其不同平台和不同程序分析粒度下的泛化能力仍有待进一步系统验证。

（五）TESSERACT: Eliminating Experimental Bias in Malware Classification across Space and Time (Security'19)

1. 研究动机

尽管 Android 恶意软件分类领域的众多研究报告了接近完美的检测性能（F1 分数高达 0.99），但这些结果往往建立在不同的数据集、不同的特征工程策略和不同的评估设置之上，使得不同研究之间的性能无法直接比较。该工作的核心动机并非开发新的检测模型，而是系统性地识别并量化实验偏差对恶意软件分类性能评估的影响。作者认为，社区尚未真正理解：报告的高性能究竟源于方法本身的优越性，还是源于存在偏差的实验设计。

2. 研究挑战

该研究面临的核心挑战在于实验控制与公平比较。恶意软件分类领域中大量看似优异的结果，往往建立在一组隐含且未经统一验证的假设之上。首先，时间偏差（Temporal Bias）广泛存在：使用 k-fold 交叉验证会导致训练集包含未来样本的知识，违反真实部署中“训练数据必须时间先于测试数据”的约束；同时，良性软件与恶意软件若来自不同时间窗口，模型可能学习到时间相关的伪特征而非真正的恶意行为模式。其次，空间偏差（Spatial Bias）同样严重：测试集中恶意软件比例若远高于真实环境（如 90%vs10%），会人为提升精确率等指标；训练集中的类别比例也会影响决策边界的位置。这些偏差共同导致实验室评估结果与真实部署性能之间存在巨大鸿沟。

3. 解决挑战的具体思路

为系统性消除这些偏差，作者提出了一套完整的评估框架 TESSERACT，包括三个核心约束、一个新指标和一个调优算法。三个约束分别为：（C1）时间训练一致性约束，要求所有训练样本的时间戳严格早于测试样本；（C2）时间窗口一致性约束，要求同一测试时间槽内的良性和恶意样本来自相同时间窗口，避免模型学习时间伪特征；（C3）真实恶意软件比例约束，要求测试集中恶意软件比例接近真实环境估计值（如 Android 环境约为 10%）。在此基础上，作者提出了 AUT（Area Under Time）指标，通过计算性能曲线在时间维度上的积分，将模型在多个时间点的表现压缩为单一数值，便于公平比较不同方法的时间鲁棒性。此外，作者还设计了训练比例调优算法，通过网格搜索找到最优的训练集恶意软件比例，在满足错误率约束的前提下最大化目标性能（如 F1、精确率或召回率）。

4. 最终效果

实验结果揭示了实验偏差的严重性及 TESSERACT 框架的有效性。在相同数据集上，使用传统 10-fold 交叉验证的 ALG1 和 ALG2 模型 F1 分数分别达到 0.91 和 0.97，但在消除偏差后，其 AUT(F1, 24m)分别降至 0.58 和 0.32，性能下

降高达 50%。更重要的是，TESSERACT 揭示了反直觉的结果：深度学习方法 DL 虽然在传统评估中表现一般，但其 AUT 达到 0.64，成为时间鲁棒性最强的方法。通过训练比例调优，可在保持清洁样本错误率小于 10%的前提下，将 ALG1 和 DL 的恶意样本 F1 分数从 0.58 和 0.64 分别提升至 0.62 和 0.65。这些发现表明，许多现有研究报告的性能显著高估了模型在真实部署中的表现。

5. 不足与改进方向

TESSERACT 虽然系统性地识别并量化了实验偏差，但其本身并不直接解决概念漂移或标签噪声等根本性问题。该工作假设研究者能够获取高质量、时间戳准确且标注可靠的数据集，这在真实工业环境中并不总是成立。此外，AUT 指标虽然提供了统一的性能度量，但其对时间粒度（如月度 vs 季度）的选择较为敏感，不同粒度可能导致不同的排序结果。尽管如此，该工作为 Level-2 研究奠定了坚实的方法论基础：在讨论模型鲁棒性与泛化能力之前，必须首先建立无偏差的评估基准。TESSERACT 框架及其开源实现为后续标签噪声、概念漂移等问题的研究提供了可信的评价标准，使得不同方法之间的比较更加科学和公正。

（六）Functionality-preserving Black-box Optimization of Adversarial Windows Malware (TIFS 2021)

1. 研究动机

基于机器学习的 Windows 恶意软件检测器在黑盒场景下容易受到对抗样本攻击。然而，现有攻击方法存在三个主要缺陷：（i）查询效率低，依赖于对输入恶意软件迭代应用随机转换；（ii）需要在优化过程的每次迭代中在沙箱内执行对抗性恶意软件，以确保其恶意功能得以保留，这一验证步骤极大增加了计算开销；（iii）对输入样本的修改量未加限制，可能产生文件大小异常的对抗样本，易被简单规则检测。该工作旨在克服这些局限，提出查询高效、功能保持且更隐蔽的黑盒攻击方法。

2. 研究挑战

首先，如何在不破坏恶意软件功能的前提下修改样本是核心难题。许多现有方法要么在特征空间操作（无法生成真实可执行文件），要么使用可能破坏功能的转换（需要沙箱验证）。其次，查询效率受限于随机扰动策略——注入随机字节往往需要大量尝试才能逃避检测。再次，如何在逃避检测与保持隐蔽性之间取得平衡是另一挑战：盲目增加扰动大小虽能提升逃避率，但会产生文件大小异常、节数过多等可疑特征。最后，攻击需要在仅能获取模型输出（甚至仅标签）的黑盒条件下有效，这排除了梯度等白盒信息的利用。

3. 解决挑战的具体思路

作者提出 GAMMA（Genetic Adversarial Machine learning Malware Attack）框架，通过遗传算法优化对抗性恶意软件。该框架的核心创新在于三方面。

首先，使用功能保持的结构性操纵：仅在 PE 文件末尾填充（padding）或注入新节（section injection），这些操作利用文件格式的冗余空间，注入的内容永不执行，从而无需沙箱验证功能完整性。

其次，注入内容并非随机字节，而是从良性程序中提取的真实节内容。实验中随机选取 75 个 .rdata 节作为候选内容池（总计最大约 2.5 MB），这使得对抗样本的字节分布更接近良性软件，大幅提升逃避效率。

再次，将攻击形式化为约束优化问题：

$$\min_{s \in S} F(s) = f(x \oplus s) + \lambda \cdot C(s), \text{subject to } q \leq T$$

其中目标函数 $F(s)$ 同时最小化分类得分 $f(x \oplus s)$ 和惩罚注入大小 $C(s)$ ，参数 $\lambda > 0$ 控制两者权衡。惩罚项定义为 $C(s) = c^T s$ ，其中 $c \in \mathbb{R}^k$ 是记录各良性节大小的向量， $c \in \mathbb{R}^k$ 表示从 k 个候选节中提取内容的比例。约束 $q \leq T$ 限制查询次数上界。

遗传算法通过选择（selection）、交叉（crossover）和变异（mutation）迭代优化节选择向量 s 。实验中设置种群大小 $N=10$ ，每次迭代产生 10 个候选并查询模型 10 次，查询预算 T 在 10 到 510 之间变化。当达到查询预算或连续 5 轮目标函数未显著下降时，算法收敛并返回最优对抗样本。

该方法还支持硬标签场景（仅返回类别）：将未逃避样本的 $f(x) = \infty$ 以过滤之，优先寻找逃避样本后再通过惩罚项 $C(s)$ 优化负载大小。

4. 最终效果

在 MalConv (CNN) 和 GBDT 两个流行检测器上, 使用 500 个恶意样本进行测试, GAMMA 显著优于随机扰动基线。两个检测器的初始性能分别为: GBDT 在阈值 0.8336 下 TPR=95% (FPR=3.9%), MalConv 在阈值 0.5 下 TPR=69% (FPR=3.5%)。

以 $\lambda = 10^{-6}$ 为例, 经过 500 次查询:

-Padding 攻击: GBDT 检测率降至 33% (逃避率 67%), 注入负载 413 KB; MalConv 检测率降至 6% (逃避率 94%), 负载 511 KB。

-Section injection 攻击效果更佳: GBDT 检测率降至 22% (逃避率 78%), 负载 643 KB; MalConv 降至 10% (逃避率 90%), 负载 487 KB。

对比之下, 注入相同大小的随机字节后检测率几乎无变化 (图 4 中 Random 曲线), 验证了从良性程序提取内容的有效性。

硬标签场景: 即使模型仅返回类别 (不返回置信度), GAMMA 在 500 次查询内仍能达到类似效果。例如 $\lambda = 10^{-6}$ 时, GBDT 检测率降至 30%, MalConv 降至 5%。

迁移性实验: 将针对 GBDT 优化的 section injection 对抗样本 (200 个) 上传至 VirusTotal, 平均检测引擎数从 46.56 降至 34.50 (平均逃避 12.06 个引擎)。在 9 个 2019 年 Gartner 魔力象限的顶级杀毒软件中, 部分检测率从 93.5% 骤降至 30.5% (AV1), 从 85.0% 降至 43.5% (AV3)。

计算开销: 每次查询耗时为 GBDT 约 0.60 ± 0.24 秒, MalConv 约 0.86-0.93 秒, 500 次查询总计约 5-8 分钟, 远快于需要沙箱验证的方法 (Castro 等人报告每次迭代约 4 分钟)。

5. 不足与改进方向

该工作的局限性主要体现在防御视角的缺失和适用范围的限制。首先, 论文仅展示攻击有效性, 未探讨任何防御措施, 如使用对抗训练、鲁棒特征或异常检测等缓解策略。其次, 攻击仅针对基于静态分析的检测器有效, 对采用动态分析 (执行样本提取行为特征) 的检测器无效, 因为注入的良性内容永不执行, 不会

改变运行时行为。再次，虽然文件格式支持更多功能保持操纵（如修改导入表、DOS 头等），但实验仅评估 padding 和 section injection 两种，未充分验证框架的通用性。最后，对抗样本的隐蔽性评估仅限于文件大小，未考虑节名称、熵值等更细粒度的统计特征可能暴露攻击痕迹。从 SoK 视角看，该工作代表了 Level-2 中对抗攻击研究的一个重要方向：它揭示了即使在黑盒、功能约束的严格条件下，精心设计的优化算法仍能高效生成对抗样本，强调了静态检测器面临的现实威胁，为后续防御研究（如鲁棒特征设计、异常检测）提供了重要参照。

（七）Make your IoT environments robust against adversarial machine learning malware threats: a code-cave approach (SDIoTSec 2024)

1. 研究动机

随着 IoT 设备的快速普及，深度学习驱动的恶意软件威胁狩猎模型（如 MalConv）在工业 IoT 环境中得到广泛部署。然而，这些模型对对抗机器学习攻击的鲁棒性尚未得到充分验证。现有对抗攻击研究多聚焦于 Windows PE 文件或 Android APK，针对 IoT 环境中广泛使用的 Linux ELF 可执行文件的攻击方法较少。同时，现有方法往往需要修改大量字节（如 2000 字节）才能实现约 10% 的逃避率，这在资源受限的 IoT 设备上易被检测。因此，本研究旨在提出一种针对 IoT 恶意软件检测模型的、更隐蔽且高效的对抗攻击方法。

2. 研究挑战

首先，生成对抗样本时必须在保持恶意功能与最小化扰动之间取得平衡。一方面，过度修改可能破坏恶意软件功能；另一方面，扰动过大会降低隐蔽性，增加被检测风险。其次，IoT 可执行文件(ELF 格式)的结构与 Windows PE 或 Android DEX 不同，需要重新识别可安全修改的攻击面。再次，现有方法多依赖随机字节注入，效率低下且效果有限。最后，在资源受限的云边 IoT 设备上，攻击必须计算高效，避免过多迭代。

3. 解决挑战的具体思路

该工作提出了一种基于代码洞穴（code-cave）的对抗攻击方法。核心思想是利用 ELF 文件中未使用的空间（如.data 节中的填充区域）注入对抗性负载，而非修改.text 节（可能破坏功能）或随机填充。该方法的优化目标可形式化为：

$$\max C(M, x', y) \text{ subject to } x' = x + \delta_x$$

其中要求扰动满足： $|\delta_x|_d \leq \varepsilon \cdot |x|$ ，以最小化修改幅度。

具体步骤如下：

（i）攻击面识别（Algorithm 1）：扫描 ELF 文件的.text、.rodata、.data 三个节，使用代码洞穴检测算法找到大小超过阈值（如 256 字节）的连续零字节区域，记录其起始和结束地址；

（ii）对抗负载生成（Algorithm 2）：使用 FGSM（Fast Gradient Sign Method）计算梯度方向：

$$\delta_x = \varepsilon \cdot \text{sign}(\nabla_x C(M, x', y))$$

但不直接修改整个文件，而是仅在检测到的代码洞穴中注入该扰动；

（iii）迭代优化：将注入负载后的样本 x' 输入 MalConv，若预测概率未超过阈值（如 0.5），继续调整负载直至逃避成功。

该方法的关键创新在于将梯度驱动的扰动与文件格式约束结合，确保修改集中在不影响执行的冗余空间。

4. 最终效果

在 IoT 恶意软件数据集上的实验表明，该方法显著优于基线。首先，攻击有效性方面：针对 MalConv，通过代码洞穴注入实现约 16% 的逃避率（检测准确率保持在 98%）；针对 Multi-kernel SVM 模型，逃避率进一步提升至 24%（检测准确率保持在 99%）。

5. 不足与改进方向

该工作的局限性主要体现在三个方面。首先，攻击仅针对静态分析模型有效，对基于动态行为（如系统调用序列）的检测器无效，因为注入的字节永不执行。

其次，实验仅在 MalConv 和 Multi-kernel SVM 上验证，未评估在其他架构（如基于 Transformer 的检测器）或其他操作系统（如 Windows、Android）上的迁移性。再次，逃避率仍有提升空间：16%-24%的逃避率虽优于基线，但距离实用攻击（如>50%）仍有差距；作者提出未来可探索自适应负载生成或结合其他操纵技术（如混淆）提升效果。最后，论文未讨论防御措施，如检测代码洞穴异常或使用对抗训练增强鲁棒性。从 SoK 视角看，该工作填补了 IoT 恶意软件对抗攻击研究的空白，强调了代码洞穴作为独特攻击面的重要性，为后续防御研究（如检测未使用空间的异常修改）提供了启示，并突显了在资源受限环境下平衡攻击效率与隐蔽性的技术挑战。

（八）PBP: Post-training Backdoor Purification for Malware Classifiers (NDSS 2025)

1. 研究动机

近年来，后门投毒攻击成为机器学习恶意软件分类器面临的新兴威胁。攻击者通过在公共恶意软件库中注入带有隐秘触发器的样本，污染训练数据，导致模型在遇到特定触发器时将恶意软件误判为良性。现有防御方法主要依赖集成模型输出的分歧来检测投毒样本，但这些方法不适用于两类现实场景：（i）用户从第三方（如 MLaaS 平台）获取已训练模型后，无法访问完整训练数据或训练过程；（ii）用户希望在训练后阶段（如发现模型异常行为后）移除后门。因此，亟需一种后训练（post-training）防御方法，能够在仅使用少量清洁数据的条件下，有效净化已被投毒的恶意软件分类器。

2. 研究挑战

首先，后门攻击的隐蔽性极强：在清洁数据上模型表现正常（准确率>98%），攻击成功率（ASR）却可高达 100%，且触发器设计多样（如基于解释引导的 Explanation-guided 或基于家族的 Jigsaw Puzzle），无统一检测模式。其次，现有后训练防御（如简单微调、线性探测）在低投毒率（ $\leq 0.5\%$ ）场景下几乎无效，ASR 仍维持在 99%以上。再次，防御必须在不假设触发器类型、不访问完整训

练数据、不知道投毒率的严格约束下进行。最后，净化过程需平衡两个冲突目标：最大化降低 ASR 的同时，最小化对清洁样本准确率（C-Acc）的损害。

3. 解决挑战的具体思路

PBP 提出了一种基于激活分布偏移的两阶段净化策略。

第一阶段：神经元掩码生成

(i) 初始化噪声模型 $\tilde{\theta}$ ，使用清洁微调数据 D_{ft} 和对齐损失 $\mathcal{L}_{align}$ 训练，该损失强制 $\tilde{\theta}$ 的每层激活分布（均值 μ 、方差 σ^2 ）对齐后门模型 θ_0 的批归一化（BN）统计量，同时优化分类损失 $\mathcal{L}_{ce}$ 。对齐损失定义为：

$$\mathcal{L}_{align} = \sum_{l=1}^n \|\mu(\theta_l \circ I^l) - \mu(\tilde{\theta}^l I^l)\|_2 + \|\sigma^2(\theta_l \circ I^l) - \sigma^2(\tilde{\theta}^l I^l)\|_2$$

总损失为：

$$\mathcal{L}_{re} = \mathcal{L}_{ce} + \alpha \cdot \mathcal{L}_{align}$$

(ii) 训练过程中，计算每个神经元对总损失 $\mathcal{L}_{re}$ 的梯度 L_2 范数，选择 Top-K 梯度最大的神经元作为后门神经元集合 N_m ：

$$N_m = \arg \max_k \|\nabla_{\theta} \mathcal{L}_{re}(\tilde{\theta})\|_2$$

第二阶段：激活偏移微调

(i) 对后门模型参数添加高斯噪声扰动初始状态：

$$\theta'_0 = \theta_0 + \mathcal{N}(0, \sigma^2 I)$$

(ii) 使用掩码梯度优化策略，定义掩码向量：

$$m_{\theta,i} = \begin{cases} -1, & i \in N_m \\ 1, & \text{otherwise} \end{cases}$$

更新规则为：

$$\theta_{t+1} = \theta_t - \eta \odot m \odot \frac{\partial \mathcal{L}_{ce}(f_{\tilde{\theta}}(x), y)}{\partial \theta_t}$$

(iii) 采用交替优化策略，奇数轮应用掩码梯度，偶数轮正常更新，避免良性任务性能退化。

该方法的核心直觉是：后门神经元在处理带触发器样本时激活分布显著偏移，通过反向优化这些神经元，可将其激活"拉回"正常分布，从而消除后门功能。

4. 最终效果

在 EMBER (Windows PE) 和 AndroZoo (Android) 两个数据集上, 针对 Explanation-guided (非家族目标) 和 Jigsaw Puzzle (家族目标) 两类 SOTA 后门攻击的实验表明, PBP 显著优于所有基线。

EMBER 数据集 (非家族目标攻击): 当投毒率为 0.5%、微调数据占比 10% 时, 传统微调方法 (FT) ASR 仍维持在 99.5%, 而 PBP 将 ASR 降至 17.83% (降低 81.67 个百分点), C-Acc 仅从 99.01% 降至 96.57% (下降 2.44 个百分点), DER 达到 0.81。当投毒率为 1% 时, PBP 将 ASR 从 99.54% 降至 15.44%, C-Acc 从 99.06% 降至 96.52% (下降 2.54 个百分点)。

AndroZoo 数据集 (家族目标攻击): 当投毒率为 0.5% 时, PBP 将 ASR 从 82.91% 降至 3.83% (降低 79.08 个百分点), C-Acc 从 98.53% 降至 96.76% (下降 1.77 个百分点)。更严苛的低微调数据场景 (仅 1% 训练数据, 约 923 样本) 下, PBP 仍能将 AndroZoo 的 ASR 降至 20% 以下, 而基线方法 (如 FT-Init、FE-tuning) ASR 几乎无变化。

值得注意的是, PBP 对家族目标攻击 (Jigsaw Puzzle) 净化效果更佳, 在投毒率为 5% 时, ASR 可从 99.72% 降至 0.89% (降低 98.83 个百分点), 这与该攻击对微调更脆弱的特性相符。迁移性方面, 图 5 展示 PBP 净化后的模型在带触发器样本上的激活分布与清洁模型几乎一致, 而其他微调方法仍保留明显偏移。此外, PBP 对超参数 ($\alpha \in [0.001, 0.01]$ 、 $\sigma \in [0.5, 1.0]$) 和模型架构 (从 20M 到 120M 参数) 表现出良好鲁棒性。

5. 不足与改进方向

PBP 的主要局限体现在以下方面。首先, 清洁准确率下降 (约 2-3 个百分点) 不可避免, 这源于净化过程扩大了决策边界以区分后门样本, 但可能导致部分良性样本被误判为恶意 (假阳性上升); 作者通过额外 20 轮微调可部分恢复准确率 (从 96.76% 提升至 98.18%, 提升 1.42 个百分点), 但仍未完全解决该权衡。其次, 方法假设微调数据完全清洁, 若微调数据本身含投毒样本, 效果可能受影响 (尽管实验显示对微调数据的家族分布、类别比例容忍度较高)。再次, 实验

主要在 MLP 架构上验证，虽理论上兼容 CNN、ResNet 等（因基于梯度稀疏性假设），但缺乏充分实证。最后，论文未探讨自适应攻击（如对抗 PBP 优化触发器）的鲁棒性。

从 SoK 视角看，PBP 代表了 Level-2 后训练防御的重要进展：它首次系统性解决了恶意软件分类器的后门净化问题，无需知晓攻击细节，仅需 1% 清洁数据即可将 ASR 从接近 100% 降至极低水平（如从 99.72% 降至 0.89%，实现 100 倍以上的改进），为 MLaaS 场景下的模型安全使用提供了实用方案，并为未来研究提出了新方向（如结合逆向工程精准定位后门神经元、探索自适应攻击防御等）。

（九）对比与总结

● 横向对比

论文标题	研究问题	研究方法	优势	局限性
Decoding the Secrets (CCS 2023)	检测性能的影响因素	构建大规模平衡数据集 对比静态特征、动态特征、分类器性能	1.静态特征优于动态特征，结合仅边际提升 2.家族数量越多分类越难，样本均衡提升泛化性	1.未直接解决概念漂移 / 标签噪声 2.动态特征易受行为缺失影响
Differential Training (NDSS 2021)	标签噪声	双模型训练 + 损失向量离群点检测，识别并修正噪声标签	1.通用适配各类检测模型 2.接近全正确标签	1.依赖模型训练的中间状态 2.极端噪声场景效果未验证
Continuous Learning (Security 2023)	Android 概念漂移	分层对比学习 + 伪损失不确定性采样 + 主动学习持续更新模型	1.漏检率从 14% 降至 9%，误报从 0.86% 降至 0.48% 2.7 年数据上性能更稳定	1.仍依赖人工标注 2.未覆盖 Windows 场景
Revisiting Concept	Windows	控制流图 +	1.少量标注	1.依赖静

Drift (NDSS 2025)	概念漂移	GNN + 对抗域自适应, 学习漂移不变特征	样本即可适配漂移 2.2024 真实样本流上检测效果显著 优于 SOTA	态分析工具 构建 CFG 2. 多平台适配性待验证
TESSERACT (Security '19)	实验偏差 (时间/空间偏差)	时间约束切分 + AUT (时间下涵盖率) 指标	1. 消除 F1 虚高, 揭示真实性能 2. 证明深度学习比传统 ML 更抗老化	1. 是一种评估框架而非检测算法 2. 揭示了现有模型性能随时间衰减的残酷事实
Functionality-preserving (TIFS '21)	黑盒对抗攻击 (功能保持)	遗传算法优化 + 良性 Payload 外部注入 (Padding)	1. 查询高效, 无需沙箱验证功能 2. 黑盒条件下可绕过主流杀软	1. 增加文件体积, 易被基于文件大小的规则发现 2. 注入在末尾或新节, 结构改动较明显
Code-Cave Attack (NDSS Workshop '24)	IoT 对抗攻击 (体积/结构受限)	梯度计算 (FGSM) + 代码空洞 (Code Cave) 内部填充	1. 不增加文件体积, 维持原有结构 2. 针对资源受限的 IoT 环境隐蔽性极强	1. 攻击成功率严格受限 于二进制文件中可用空洞的大小 2. 需精细解析 ELF 格式
PBP (NDSS '25)	后门防御 (净化) (无训练数据场景)	梯度追踪定位神经元 + BN 层统计量调整 (修正激活	1. 仅需 1% 良性数据, 无需访问原训练集	1. 仅适用于后训练 (Post-training) 阶段

		分布)	2. 直接净化黑盒模型 (MLaaS 场景)	2. 主要针对神经网络架构
--	--	-----	------------------------	---------------

评估与认知校准路线以 **Decoding the Secrets of ML in Malware Classification** 与 **TESSERACT** 为代表。这类工作通过受控的大规模测量研究，揭示数据集构造、实验设计对性能评估的决定性影响，为后续研究提供了可靠的评价坐标系。**TESSERACT** 进一步系统化地识别时间与空间偏差，提出约束与指标，确保评估结果能够反映真实部署性能。在讨论鲁棒性之前，我们必须首先消除实验偏差与认知错位。

数据层鲁棒性路线 **Differential Training** 关注点从评估层面转向训练数据本身。该类方法将标签噪声视为影响判别模型可靠性的核心因素，并通过分析模型训练动态来自动识别并修正错误标签。该工作在不改变模型结构的前提下，通过修数据即可显著提升检测性能与稳定性。

时间维度鲁棒性路线 **Continuous Learning for Android Malware Detection** 明确接受分布随时间变化这一现实前提，并将模型更新视为一个持续过程。该类方法通过主动学习与人工标注闭环，使模型能够在长期运行中不断修正判别边界，但也不可避免地引入了对人工标注资源的依赖。

表征不变性学习路线以 **Revisiting Concept Drift in Windows Malware Detection** 为代表，该类方法通过域自适应与不变特征学习，试图在表征层面抽象出跨时间稳定存在的判别依据，从而减少对持续人工干预的依赖，体现了 **Level-2** 方法在理论深度上的一次提升。

对抗攻击与防御路线 **Functionality-preserving**、**IoT Code-Cave** 和 **PBP** 代表了从攻击到防御的完整链条。前两者揭示了判别式检测器在精心设计的对抗样本面前的脆弱性：通过功能保持的内容注入或代码洞穴利用，攻击者能以极小的修改（<1MB 或 256-512 字节）高效逃避检测，且具有跨平台迁移性。**PBP** 则从防御视角出发，针对后门投毒这一更隐蔽的威胁，提出后训练净化方案，仅需 1% 清洁数据即可将 ASR 从 100% 降至 1%，为 MLaaS 等现实场景提供了实用防护。这些工作共同强调了在对抗环境下构建鲁棒检测器的紧迫性与挑战性。

● 纵向演进

从纵向视角审视这些工作，可以清晰地看到 Level-2 研究对"鲁棒性"这一概念的理解正在持续升级。早期研究往往将恶意软件检测视为一个静态分类任务，重点放在单一数据集上的性能最优化；随后，TESSERACT 等工作开始意识到实验偏差对结果的系统性影响，强调评估方法论的重要性；进一步地，Differential Training 揭示了标签噪声这一数据层面的挑战；再进一步，时间维度被引入核心建模对象，Continuous Learning 和 Revisiting Concept Drift 将概念漂移视为无法回避的现实问题，并分别从持续学习和不变特征学习两个角度给出应对方案；最后，对抗攻击与防御研究（Functionality-preserving、IoT Code-Cave、PBP）将视角扩展至敌对环境，揭示了恶意样本操纵（对抗攻击）和训练数据投毒（后门攻击）对模型的严重威胁，并探索相应防御机制。

这一演进路径体现了从"理想静态假设"到"现实动态对抗"的范式转变：研究者逐渐认识到，真实部署中的恶意软件检测器不仅要在清洁、平衡、静态的数据集上表现优异，更要在时间漂移、标签噪声、实验偏差、对抗样本和后门攻击等多重挑战下保持稳定的判别能力。

● 统一的建模视角

具体而言，TESSERACT 通过约束实验设计，消除时间与空间偏差，确保评估结果的可信性；Differential Training 通过识别并修正错误标签，约束模型对不可靠标签信号的依赖，从而提升数据层面的可信性；Continuous Learning 通过持续更新与样本选择机制，约束模型对过时时间相关性的依赖，使其能够跟随分布变化调整判别边界；Revisiting Concept Drift 则通过域自适应与不变特征学习，显式约束模型对域特异统计特征的依赖，引导其关注跨时间稳定存在的行为与结构模式；Functionality-preserving 和 IoT Code-Cave 从攻击者视角揭示了模型对输入扰动的脆弱性，强调了功能保持约束下对抗样本生成的可行性；PBP 则从防御者视角出发，通过激活分布对齐与掩码梯度优化，约束模型对后门触发器的依赖，实现训练后净化。

从这一统一建模视角出发，可以看到不同工作虽然在技术实现上差异显著，

但其核心目标高度一致：抑制那些偶然成立、短期有效、却无法在长期部署或对抗环境中持续成立的统计相关性。无论是实验偏差、标签噪声、概念漂移、对抗扰动还是后门触发器，本质上都是对模型泛化能力的"虚假捷径"——模型依赖这些不稳定信号获得高训练性能,但在真实复杂场景下迅速失效。Level-2 的研究进展，正是围绕识别、量化并消除这些"虚假依赖"展开，逐步构建起更加可信、鲁棒、可持续部署的判别式恶意软件检测系统。

三、Level 3：走向“理解”恶意软件

尽管判别式模型在恶意软件“检测”和“分类”任务上取得了显著成功，但它们的局限性主要体现在对恶意软件深层语义和行为逻辑的理解上。这些传统模型通常专注于表面特征，如文件特征、API 调用、执行路径等，无法深入挖掘恶意软件的复杂行为和隐藏意图，在面对复杂、未知的恶意软件时，往往表现不佳。基于此，相关研究转向利用大语言模型（Large Language Model, LLM）强大的代码理解和生成能力进行恶意软件的语义恢复和可解释分析。

（一）ReSym: Harnessing LLMs to Recover Variable and Data

Structure Symbols from Stripped Binaries

1. 研究动机

二进制反编译技术广泛应用于恶意软件分析和漏洞检测等领域。传统的反编译工具虽然能够恢复程序的控制结构和局部变量，但通常不能恢复有意义的变量名，且恢复的类型往往不准确。现有的方法虽然在某些方面取得了进展，但仍然存在诸多限制。

2. 挑战

- 1) **变量名和类型恢复的挑战：**在二进制反编译过程中，变量名和类型信息会丢失，现有工具仅能恢复一些简单的变量名和类型信息。
- 2) **用户定义数据结构的恢复：**现有方法通常只能恢复一些简单的数据类型和结

构，但对于用户定义的数据结构例如数组和内嵌结构时，恢复的准确性和完整性都较差。

- 3) **LLM 应用的局限性：**虽然 LLM 在自然语言处理和代码理解方面表现优异，但直接使用 LLM 进行符号恢复时会面临反编译代码的特殊性、LLM 输入长度的限制和幻觉等问题。

3. 解决挑战的具体思路

论文提出了 ReSym，一种结合 LLM 和程序分析的混合方法，旨在恢复二进制文件中的符号信息，特别是用户定义的数据结构。主要思路如下：

- 1) **任务分解：**论文将符号恢复任务分解为两个子任务。第一个子任务是恢复局部变量的符号（变量名和类型）。第二个子任务是恢复用户定义的数据结构（如结构体）及其字段信息。
- 2) **LLM 微调：**为了适应符号恢复任务，论文对两个不同的 LLM（VarDecoder 和 FieldDecoder）进行了微调，分别处理局部变量和数据结构的恢复。
- 3) **Prolog 推理系统：**由于 LLM 存在令牌长度限制，无法处理整个程序的上下文，ReSym 引入了基于 Prolog 的推理算法来解决这个问题。推理系统负责将多个函数的局部结果进行交叉验证，并进行全局聚合，确保恢复的符号信息在全局上下文中是一致的。推理系统还使用了相似度偏置投票来解决 LLM 输出中可能存在的冲突。
- 4) **多次 LLM 查询和局部推理：**为了规避 LLM 输入长度限制，ReSym 采用了多次查询的方式，每次查询单个函数的反编译代码，并通过推理系统将这些局部信息进行聚合和整合，恢复出全局的符号信息。

4. 最终效果

论文构建了一个包含 7416 个 C/C++ 项目的大规模的公开数据集用来训练和验证 ReSym 模型的有效性。实验结果表明，ReSym 在恢复变量名和类型方面，取得了 56.4% 和 64.6% 的准确率，在数据结构恢复方面达到了 72.9% 的精度，尤其在处理内联结构或被分解为多个变量的数据结构时，ReSym 的表现非常优异，

F1 分数高达 85.9%。

5. 不足与改进方向

尽管 ReSym 取得了显著的效果，但该方法仍存在一些不足之处。首先，尽管 ReSym 在用户定义的数据结构恢复上表现优异，但对于某些极为复杂或者非常特殊的数据结构，恢复的效果仍有提升空间。其次，虽然 ReSym 通过分解任务和使用 Prolog 推理系统有效减少了计算开销，但在面对大规模的二进制文件时，推理阶段仍可能面临一定的性能瓶颈，尤其是在复杂程序的推理过程中。最后，ReSym 的性能依赖于其训练数据集的质量和多样性，某些未在训练数据中出现的特殊数据结构或程序模式可能会影响恢复效果。

未来可以考虑扩展训练数据集，覆盖更多样化的编程模式和数据结构。目前的研究主要集中在 C/C++ 代码的恢复上，未来可以扩展到其他编程语言，提升 ReSym 的通用性；可以探索更高效的推理机制，减少 Prolog 推理带来的计算开销，提升系统的实时性。

(二) MalGuard: Towards Real-Time, Accurate, and Actionable Detection of Malicious Packages in PyPI Ecosystem

1. 研究动机

Python 包管理平台 PyPI 作为开源软件开发的重要组成部分，其恶意包攻击检测也逐渐成为安全问题的焦点。传统的恶意包检测方法在准确性、可解释性和实时性等方面仍存在挑战。同时，已有基于 LLM 的方法也面临计算成本高昂的问题。

2. 挑战

- 1) **依赖人工的特征提取：**传统的恶意包检测方法往往依赖专家手动分析恶意包，提取特征进行建模，这一过程既繁琐又容易遗漏新兴的恶意行为。
- 2) **恶意包检测的可解释性：**现有的检测方法往往缺乏足够的可解释性，检测出

的恶意包的具体行为难以追溯。安全专家在审查和验证恶意包时需要更多的透明度来理解恶意行为的根源。

- 3) **计算成本和实时性：**大语言模型虽然在恶意包检测中展示了较高的准确性，但其高计算成本和长时间的推理过程使得实时检测变得不切实际。如何在保证准确度的同时降低计算成本，是亟待解决的问题。

3. 解决挑战的具体思路

论文提出了 MALGUARD，一种结合了图中心性分析和机器学习模型，同时利用大语言模型提升可解释性的方法。具体思路如下：

- 1) **自动化特征提取：**为了避免手动提取特征，论文首先使用静态分析构建每个恶意包的 API 调用图。然后，通过计算 API 节点的中心性值（如度中心性、接近中心性等），自动选出在恶意包中最敏感的 API。
- 2) **LLM 辅助分析与解释：**为了选出的敏感 API，论文通过 LLM 来分析每个 API 的潜在恶意行为从而进行筛选（但仍需人工介入）。
- 3) **集成 LIME 增强可解释性：**为了进一步提高检测过程的透明度，论文集成了 LIME 算法用于提供关于恶意包的详细解释。

4. 最终效果

论文通过与现有的六种最先进的恶意包检测方法进行对比，展示了 MALGUARD 在准确性、召回率和精度上的优势。MALGUARD 成功识别了 113 个之前未被发现的恶意包，其中 109 个已被 PyPI 官方移除，证明了其实际应用中的有效性。MALGUARD 能够在五周内对 64,348 个新上传的包进行检测，每个包的检测时间不到 1 秒。

5. 不足与改进方向

尽管 MALGUARD 在检测性能上取得了显著进展，但仍然存在一些不足之处，未来可以进一步改进。首先，MALGUARD 依赖于已有的敏感 API 特征集，但随着攻击手法的不断演化，新的恶意行为可能不在现有特征集中。这要求系统

不断更新和扩展特征集，可能需要更多的自动化和动态更新机制。其次，虽然 LLM 在本研究中仅用于特征精炼和解释生成，但它仍然可能生成不准确的恶意行为描述（即幻觉问题）。尽管有人工验证机制，但这一问题仍然可能影响最终的检测效果。最后，目前 MALGUARD 的方法主要依赖于 API 调用图，针对具有复杂代码结构（如混淆代码、跨文件攻击等）的恶意包可能存在一定的局限性，未来可以通过结合更多的动态分析手段来提升检测精度。

（三）Leveraging Large Language Models to Detect npm Malicious Packages

1. 研究动机

随着软件供应链攻击的日益增多，npm 成为了攻击者的重要目标。静态分析、机器学习和规则检测等恶意代码检测技术往往存在高误报率、低准确性和无法应对复杂攻击类型的局限性。传统的恶意代码检测方法需要多个工具的集成，这不仅增加了系统的复杂性，还使得误判率较高，且难以满足对新型攻击的检测需求。

2. 挑战

- 1) **恶意代码检测的多样性和复杂性：**传统的恶意代码检测技术往往需要集成多种工具来检测不同的恶意模式，工作量大且容易导致误报和漏报。不同的攻击模式通常需要不同的检测策略，这使得现有技术面临较高的误判率和低效性。
- 2) **误报率和漏报问题：**静态分析方法通常有较高的召回率，能够发现大量恶意代码，但其精度较低，误报率较高，导致大量的手动复核工作。与此同时，动态分析、差异分析等其他方法则需要更多的计算资源和时间，且难以保证准确性。
- 3) **LLMs 在生产环境中的应用问题：**尽管 LLM 已在许多领域取得显著成果，但它们在恶意代码检测中的有效性和生产环境中的可行性仍缺乏实证研究，尤其是如何避免 LLM 的幻觉和如何处理大规模的数据和上下文问题。

3. 解决挑战的具体思路

论文提出了 SocketAI 工作流，该工作流结合了大语言模型和静态分析工具，在提高恶意代码检测准确性的同时，降低了成本和工作量。具体思路包括以下几个方面：

- 1) 多阶段工作流设计：SocketAI 工作流由初步报告生成（通过 LLM 生成初步的恶意代码检测报告）、关键报告复核（使用更严格的分析方法复审初步报告，评估其准确性，并根据反馈进一步修正）、最终报告生成（从多个复审报告中选出最可信的结论，生成最终报告）和人工回路（对高风险报告（如高恶意分数的包）进行人工复核，确保结果的可靠性）四个主要步骤组成。
- 2) 静态分析与 LLM 结合：在工作流中，首先使用静态分析工具进行预筛选，将无害文件剔除，这样减少了 LLM 需要分析的文件数量，节约了计算资源和时间。
- 3) LLM 与自我迭代技术：为了减少幻觉和提高分析的准确性，本文采用了自我迭代技术，即通过多次反馈和修正优化模型的输出。同时，利用角色扮演提示、思维链和 zero-shot 推理，使得模型能够在没有特定训练的情况下，准确推理并分析恶意代码。

4. 最终效果

与传统的静态分析工具（如 CodeQL）进行基准对比时，SocketAI 在精度和 F1 分数上表现出色。使用 GPT-4 和 GPT-3 下精度和 F1 分数均显著高于 CodeQL。通过静态分析的预筛选，减少了需要 LLM 分析的文件数量。GPT-3 的成本下降了 60.9%，GPT-4 的成本下降了 76.1%，有效降低了分析成本。SocketAI 工作流在提高恶意代码检测效率的同时，也减少了人工审查的工作量。

5. 不足与改进方向

尽管本文采用了多阶段自我迭代和角色提示词等方法来减少幻觉问题，但在某些情况下，尤其是 GPT-3 模型中，依然存在较为严重的幻觉现象（如错误地将空文件标记为恶意文件）。因此，未来可以进一步探索如何在 LLM 中引入更

多的外部知识库或推理模块，以减少模型的幻觉问题。由于 LLM 在分析长文件或跨文件的恶意代码时会受到上下文长度限制，导致无法全面分析复杂的攻击模式。未来可以考虑使用更高效的模型或优化模型架构，以增强对大规模代码库的处理能力。尽管工作流程中通过人工复核进一步确保了准确性，但这也意味着该方法仍然需要人工介入，限制了其完全自动化的潜力。未来可以研究如何进一步减少人工回路的依赖，提升全自动化检测的能力。

（四）ForeDroid: Scenario-Aware Analysis for Android Malware

Detection and Explanation

1. 研究动机

随着 Android 操作系统在全球范围内的广泛使用，恶意软件对移动设备的威胁日益严重。传统的恶意软件检测方法通常依赖于静态特征来进行恶意软件分类，但容易受到代码混淆和恶意软件反沙箱技术的影响。同时这些方法通常忽略了行为的语义信息，导致其缺乏足够的鲁棒性，特别是对于未见过的恶意软件（零日攻击）和细粒度行为的检测。

2. 挑战

- 1) **无法应对零日恶意软件：**现有方法多依赖于已标注的恶意软件样本，难以应对未知的恶意软件或零日攻击。
- 2) **细粒度行为分析：**大多数检测方法只能在应用级别进行分析，难以识别和解释细粒度的恶意行为，例如 API 调用链和具体的功能实现。
- 3) **缺乏可解释性：**许多恶意软件检测方法缺乏足够的可解释性，安全分析人员往往只能得到一个“恶意”或“良性”的标签，而无法深入理解恶意软件的行为和意图。
- 4) **场景建模不足：**现有方法多依赖手工特征和规则，缺乏对于 Android 应用程序功能场景的深入理解，无法准确识别应用程序行为中的语义不一致性。

3. 解决挑战的思路

论文提出了一个基于场景感知分析的 Android 恶意软件检测和解释框架 ForeDroid，具体思路如下：

- 1) **场景感知建模：**应用程序通常由多个功能场景组成，每个场景对应一定的功能和行为预期。恶意应用通常会违反这些场景的预期行为。ForeDroid 通过分析应用程序的行为并与预定义的场景进行对比，识别出不符合预期的恶意行为。
- 2) **敏感行为表示和检测：**ForeDroid 采用了敏感 API 调用链抽取的策略，提取从入口点到敏感 API 调用的完整路径。通过结合 LLM，ForeDroid 能够将复杂的 API 调用链转化为简洁的自然语言描述，并利用这些描述进行语义分析。
- 3) **无监督的异常检测：**为了解决零日恶意软件检测问题，ForeDroid 采用了无监督学习的方法。它通过对良性行为的建模，学习良性行为的分布，并通过对比应用程序行为的语义偏差来识别异常。
- 4) **行为解释与可解释性：**ForeDroid 不仅能够检测恶意软件，还能通过 LLM 生成可解释的报告，帮助安全分析人员理解恶意软件的行为和意图。

4. 最终效果

在对比实验中，ForeDroid 在零日恶意软件检测方面展现了优异的性能。在零日恶意软件检测中，ForeDroid 在精确度、召回率和 F1 分数上显著超越了其他基线方法。在细粒度行为分析任务中，ForeDroid 在 GPMalware 数据集上取得了 0.94 的 F1 分数，超越了现有方法，表现出其对恶意软件行为的高效识别能力。

5. 不足与改进方向

尽管 ForeDroid 在零日检测方面表现优秀，但由于其采用无监督的异常检测方法，可能会对一些模糊的行为产生较高的误报率。这是因为这些行为在功能场景上可能与恶意行为相似，但缺乏明确的恶意证据。因此，如何进一步扩展良性应用库的多样性，增强对新兴或特殊行为的识别能力，是未来需要改进的方向。ForeDroid 的实现涉及多个复杂的模块，如静态分析、敏感行为提取、LLM 摘要生成等。其工程复杂性较高，可能导致在大规模应用场景中的部署成本较高。如

何在保证检测效果的同时降低系统的复杂度和成本，是一个值得关注的问题。尽管 LLM 在总结和生成报告方面表现出色，但其在处理某些复杂恶意行为时可能会出现不一致或不准确的解释。这需要进一步优化 LLM 的生成过程，确保其输出的准确性和可信度。

（五）不同方法综合 LLM 能力的方式

方法	ReSym (CCS 2024)	MalGuard (USENIX Security 2025)	SocketAI (ICSE 2025)	ForeDroid (CCS 2025)
应用场景	C/C++	PyPI	JavaScript	Android
目标任务	符号恢复	提取特征信息	恶意代码检测	恶意行为解释
应用方式	特定子任务 下微调 LLM	提示词工程	提示词工程	提示词工程
解决幻觉问题	基于 Prolog 的推理算法 确保结果有效	人工审查特征的有效性	迭代自改进方法、思维链（CoT）	无额外处理
解决上下文长度问题	将代码划分 多个代码片 段处理，然 后将所有结 果聚合	提炼核心代码片段	只能针对单文件 分析，无法处理 多文件和大文件	输入精炼后的 完整调用链和 相关场景信息
效率	需要离线训练 2 个参数量 为 3B 的 LLM	只利用 LLM 进行 离线特征构建。利用 轻量级 ML 模型检测 恶意代码	利用静态分析预 筛选代码。分场 景使用不同规模 LLM	只利用 LLM 对 恶意分数高的 行为进行解释

从综合角度来看，这四篇论文在使用 LLM 进行代码分析和恶意行为检测时，各自采取了不同的策略和解决方案，突出表现在不同的应用场景和任务、解决幻觉和上下文和效率需求。ReSym 聚焦于 C/C++代码的符号恢复，使用 LLM 进行特定子任务的微调，通过 Prolog 推理确保结果有效。它采用分片处理的方式，先将代码划分为多个片段进行处理，最终将结果汇总。该方法需要离线训练两种 3B 参数量的 LLM。MalGuard 主要针对 PyPI 进行恶意代码特征提取，通过提示词工程和人工审查特征的有效性，利用 LLM 构建特征后，结合轻量级 ML 模

型进行恶意代码检测。SocketAI 专注于 JavaScript 代码的恶意检测，利用提示词工程和自我改进方法（思维链）来优化检测结果。其局限性在于只能分析单个文件，无法处理多个文件或大型文件。ForeDroid 主要用于 Android 应用的恶意行为解释，通过提示词工程，精炼输入数据（调用链和场景信息），然后利用 LLM 对高恶意分数的行为进行解释，不涉及额外的处理步骤。

综上所述，利用 LLM 进行恶意代码检测能够处理复杂的代码模式和恶意代码的可解释。未来，随着 LLM 训练和优化技术的不断进步，需要进一步提升对多文件、大型文件的分析能力，解决 LLM 幻觉和上下文问题，实现更加实时、精准的恶意代码检测效果。

四、Level 4：构建智能体“分析师”

Level 3 的探索证明了 LLM 在辅助分析任务上的巨大潜力，但这距离完全替代人类专家还有很长的路。Level4 的目标在于对构建能够像人类分析师一样独立工作的自主智能体进行尝试和探索。智能体不再是被动完成某个特定子任务（如反编译或代码摘要）的工具，而是能够主动规划、决策和执行整个分析流程。

（一）Large Language Models for Code Analysis: Do LLMs Really Do Their Job?

1. 研究动机

近年来，大语言模型在自然语言处理领域取得突破性进展，并在代码理解和生成任务中展现出巨大潜力。然而，现有研究缺乏对 LLMs 在混淆代码分析这一关键场景下的系统性评估，尤其是在真实世界、低可读性代码场景下的表现尚未得到充分验证。

2. 挑战

- 1) **缺乏系统性评估：**现有研究多集中于非混淆代码的理解与生成，对于 LLMs 在混淆代码分析中的能力缺乏系统性实验验证。
- 2) **混淆技术的对抗性：**恶意代码常使用控制流扁平化、字符串拆分、死代码注

入、WebAssembly 转换等技术来隐藏真实意图, 这些技术对 LLMs 构成挑战。

- 3) **模型泛化能力有限:** 小规模模型在代码分析任务中表现不佳, 难以处理长代码、复杂结构或低可读性输入。
- 4) **评价指标不完善:** 传统基于 n-gram 或 BERT 的语义相似度评估难以准确衡量代码解释的正确性, 尤其是在没有注释或上下文的情况下。

3. 解决挑战的具体思路

- 1) **构建多语言、多场景数据集:** 作者构建了包含 JavaScript、Python、C 三种语言的非混淆和混淆代码数据集, 并采用多种混淆技术 (如 Wobfuscator、控制流扁平化) 生成对抗样本。
- 2) **系统评估多类 LLMs:** 选择了 GPT-3.5、GPT-4、LLaMA-2-13B、Code-LLaMA-2-13B-Instruct、StarChat-Beta 五种代表性模型进行对比实验。
- 3) **设计多层次评估指标:** 除了传统的余弦相似度和 BERT 语义相似度外, 还引入了基于 GPT-4 的“代码解释一致性判断”评估方法, 并辅以人工验证。
- 4) **开展真实案例研究:** 选取了 Android 病毒(msg-stealer)和勒索软件(WannaCry)进行案例研究, 评估 LLMs 在恶意软件逆向工程中的实用性。

4. 最终效果

- 1) **非混淆代码分析:** GPT-4 和 GPT-3.5 表现优异, 解释准确率达 95% 以上, 能识别开源代码库、利用标识符名称进行推理; 小模型 (如 LLaMA-2-13B) 则表现较差, 常输出重复或无意义内容。
- 2) **混淆代码分析:** GPT-4 在基础混淆技术上仍保持 87% 的准确率, 但对 Wobfuscator 表现较差; 小模型几乎无法理解混淆代码。
- 3) **反混淆代码生成:** GPT-3.5 和 GPT-4 在 IOCCC 混淆代码反混淆任务中表现不佳, 仅少部分生成的代码可编译运行, 且 GPT-4 常拒绝生成代码。
- 4) **真实恶意软件分析案例:** GPT-4 能识别恶意行为模式 (如环境检测、权限请求), 但在分类时存在误报; GPT-3.5 能提取 WannaCry 的关键行为 (如端口 445 连接、服务伪装), 但忽略代码标签、保守判断。

5. 不足和改进方向

本文的混淆代码样本主要集中于 JavaScript，其他语言覆盖不足，缺乏多语言混淆技术的比较。在 LLMs 上，未包含更多开源或专用于安全的 LLMs，评估范围受限。本文虽然引入 GPT-4 作为“裁判”，但仍需人工校准，自动化程度不高。最后，LLMs 在生成可运行反混淆代码方面表现较差，尚未达到实用水平。尤其是在处理长函数或跨文件调用时，模型易忽略关键信息，记忆能力有限。

本文指出，构建更全面的混淆代码数据集、探索 LLM 记忆与泛化机理、设计更可靠的代码语义相似度度量标准、以及开发针对反混淆等任务进行优化的 LLM 微调方法，是值得探索的重要方向。

（二）Beyond Classification: Evaluating LLMs for Fine-Grained Automatic Malware Behavior Auditing

1. 研究动机

恶意软件检测已从传统的分类模型向更细粒度的行为审计发展，后者要求既要判断是否为恶意软件，又要提供可验证的行为解释与证据链。传统动态分析方法成本高且易受规避，而基于机器学习的解释方法常局限于片段级分析，缺乏可追踪性。LLMs 因其强大的代码理解和自然语言生成能力，被视为提升恶意软件审计自动化水平的潜在工具，但其在真实审计场景中的可靠性尚未得到系统评估。

2. 挑战

- 1) **细粒度标签稀缺**：缺乏函数级恶意行为标注，难以验证 LLMs 的输出是否真实反映代码意图。
- 2) **代码噪声干扰**：恶意代码常嵌入大量良性代码中，直接输入整个代码库会导致 LLMs 注意力分散。
- 3) **输出不可追踪与幻觉问题**：LLMs 在分析大代码库时易产生幻觉，输出的行为描述缺乏可验证的证据支持。
- 4) **评估体系缺失**：现有评估多集中于代码生成或分类任务，缺乏针对恶意软件

审计流程的系统化任务设计与评价指标。

3. 解决挑战的具体思路

- 1) **构建 MaIEval 评估框架：**包含可验证的真实标签（基于专家报告和敏感 API 列表）、代码上下文降噪（通过静态可达性分析）和上下文驱动的结构表示（函数签名+摘要+敏感度评分）。
- 2) **设计四阶段审计任务：**
 - 函数优先级排序：识别高风险函数；
 - 证据归因：将敏感 API 映射到具体函数；
 - 行为合成：生成结构化行为报告；
 - 样本判别：区分误分类的良性样本。
- 3) **提出复合评估指标：**包括工作负荷减少评分（WRS），结合报告质量、证据真实性和决策支持能力。

4. 最终效果

- 1) **总体表现：**Claude-3.7-Sonnet 表现最佳（WRS 50.67%），但仍远未达到可靠审计水平；小模型（如 Llama-3.1-8B）能力严重不足。
- 2) **任务表现差异：**
 - 函数优先级排序：模型普遍难以识别关键函数；
 - 证据归因：Qwen 系列覆盖率高，但 Claude 更精准；
 - 行为合成：Gemini-2.5-Flash 报告质量最高，但幻觉问题仍存；
 - 样本判别：Claude 在纠正误报方面表现最好。
- 3) **上下文重要性：**移除函数间上下文后，模型在行为合成和证据归因任务中表现显著下降。
- 4) **元数据影响：**部分模型（如 DeepSeek-V3）能从应用元数据中受益，但 Gemini-2.5-Flash 因噪声干扰表现下降。

5. 不足与改进方向

本文采用的最新恶意软件样本较少，且主要集中于 Android 平台，缺乏跨平台（如 Windows、IoT）评估。另外，使用 GPT-5-mini 作为评判者可能引入模型偏见，且未完全摆脱对人工验证的依赖。本文还发现，模型常出现“良性关联偏差”和“逻辑盲区”，难以理解复杂攻击链或滥用合法 API 的行为。大模型在长上下文处理和多轮推理中计算开销大，难以在实际 SOC 中实时部署。最后，本文未系统测试模型在面对新型混淆、多态或对抗性样本时的鲁棒性。

本文为后续研究指明了道路：将 MaIEval 扩展至多平台；探索动态分析与 LLM 的结合；开发更高效、更抗幻觉的审计专用模型架构或微调方法；以及将审计 workflow 进一步自动化、工具化，真正集成到 SOC 的管道中。

（三）DeGPT: Optimizing Decompiler Output with LLM

1. 研究动机

逆向工程中，反编译器将汇编代码转换为高级语言代码，但输出的代码质量低下：变量名无意义（如 `param_1`）、结构冗余、缺乏注释。传统基于机器学习的方法依赖大规模训练数据集，面对未见过的二进制文件时性能显著下降，且通常只解决单一问题。

2. 挑战

- 1) 反编译代码可读性差：反编译代码质量低，变量名无意义或看不出意义
- 2) 让 LLM 改代码效果不稳定、成本高
- 3) LLM 生成结果可能破坏原始程序语义：LLM 作为生成模型，其输出不具备可靠性保证，可能改变代码的原始语义，导致功能逻辑错误。
- 4) 反编译代码静态验证开销大、路径爆炸

3. 解决挑战的具体思路

DeGPT 提出了一种基于“三角色机制”的端到端优化框架，包括：

- 裁判角色 (**R_ref**)：分析反编译输出，提出优化方案（如是否需要结构简

化、变量重命名、添加注释）。

- **顾问角色 (R_adv)**：根据 R_ref 的方案，生成具体的代码修改建议。
- **操作员角色 (R_ope)**：通过“微片段语义计算” (MSSC) 检查优化后的代码是否保持原始语义，决定是否接受修改。

此外，DeGPT 还提出了 MSSC 方法，通过符号执行模拟代码片段执行路径，检查优化前后符号值是否一致，从而保证语义一致性。

4. 最终效果

DeGPT 在包含多个代码库（如 LeetCode、Coreutils、Mirai、AudioFlux）的数据集上进行了评估，结果如下：

- **结构简化**：认知负担降低 24.4%（通过 Halstead 复杂度度量）。
- **变量重命名**：30.3% 的变量名能与源代码中的变量名对应，远高于现有最佳工具 DIRTY（11.1%）。
- **注释生成**：62.9% 的注释具有实际语义价值，正确率高达 99.2%。
- **用户调研**：优化后的代码在可读性、语义理解方面显著提升，准确率提升 126%，时间节省 47%。

与 DIRTY 相比，DeGPT 在变量重命名覆盖率 (ONR) 上达到 93.6%，远高于 DIRTY 的 76.7%。

5. 不足与改进方向

MSSC 在处理复杂控制流时可能面临路径爆炸问题，尽管本文通过循环转换和超时机制缓解，但仍未根本解决；同时 MSSC 依赖随机数赋值模拟执行，理论上有极小概率出现误判，且对指针分析、类型转换等复杂场景支持有限。此外，依赖 ChatGPT 等商业 API，存在 token 成本（每函数约 198–254K tokens）和响应延迟问题。最后，尽管在未见过的代码库（AudioFlux）上表现良好，但未在更多样化、更复杂的真实恶意软件或混淆代码上进行测试。

MSSC 的路径爆炸问题可通过程序切片与增量分析优化，并增强对指针分析、复杂数据流的支持。成本控制上，可探索开源模型微调（如 Code Llama）、提

示压缩、缓存机制。功能扩展方面，可能支持跨函数优化、类型恢复、数据结构重建，并开发交互式逆向分析界面。对抗性场景中，需测试其对混淆、加壳代码的适应性，增强语义保持的鲁棒性。

（四）AV-Agent: A Bottom-Up Interpretable Malware Classifier Based on Large Language Models

1. 研究动机

恶意软件检测是网络安全的核心任务之一。传统机器学习方法（如 MalConv、GNN）在恶意软件分类中表现出色，但存在显著缺陷：模型缺乏可解释性，依赖统计模式而非逻辑推理，难以提供人类可理解的决策依据。LLM 具有强大的文本理解与推理能力，若能将其应用于恶意软件分析，有望实现“类人化”的逻辑推理与解释性分类。

2. 挑战

- 1) **文本特征选择**：如何从二进制文件中提取并文本化关键特征（如字节码、控制流图、字符串），供 LLM 理解。
- 2) **上下文窗口限制**：二进制文件特征维度高，直接输入会超出 LLM 上下文长度。
- 3) **推理能力激发**：如何设计提示工程与多阶段推理机制，使 LLM 能综合多源特征进行逻辑分类。

3. 解决挑战的具体思路

AV-Agent 提出一种分层特征构造 + 两阶段推理的框架：

1) 分层特征构造：

- 原始字节（Raw Bytes）：使用 CNN 分类器提取特征。
- 控制流图（CFG）：使用 GNN 分类器提取图结构特征。
- 启发式特征（Heuristic Features）：如字符串、API 调用等，使用传统

ML 分类器。

2) 特征文本化:

- 将分类器的置信度分数直接作为数值特征输入 LLM。
- 使用可解释性方法（如 SHAP）提取关键字符特征（如关键字串、代码段），并文本化。

3) 两阶段推理:

- Phase I: 输入三个分类器的置信度分数，LLM 进行初步分类。
- Phase II: 输入 Phase I 结果 + 关键字符特征，LLM 进行最终分类与解释。

4. 最终效果

在挖矿木马数据集（DataCon 2020）上评估，结果如下：

- 1) 分类性能: AV-Agent 在 Phase II 达到最佳 F1-score 0.9585(使用 deepseek-r1)，优于多数传统集成学习方法（如随机森林 F1=0.9572）。
- 2) 可解释性: AV-Agent 能输出类人的推理过程，如“该样本包含比特币相关字符串，疑似挖矿软件”。
- 3) 模型比较: GPT-4、Claude-3.5-Sonnet、DeepSeek-R1 等模型表现接近，推理模型（如 Claude-3.7-Sonnet）在复杂任务中表现更优。
- 4) 稳定性测试: 提示词元素排列、特征顺序对结果影响较小，模型表现相对稳定。
- 5) 成本分析: 使用 Claude-3.7-Sonnet 每样本成本约\$0.04 - 0.053，高于传统 ML 但远低于人工分析成本。

5. 不足与改进方向

当前 AV-Agent 主要依赖静态特征分析，对加壳、混淆或加密的恶意样本检测能力有限，易被传统逃避技术绕过。其框架高度依赖商用 LLM API，导致分析成本高昂且存在响应延迟，难以适应大规模实时检测场景。此外，模型仅在挖矿木马数据集上验证，泛化能力未在更广泛的恶意软件类型中得到充分评估，缺乏对动态行为和多模态特征的融合分析，限制了其在复杂对抗环境中的实用性。

提示注入等安全风险也未得到有效防护，可能影响分类可靠性。

未来可设计动态与静态特征融合的多模态分析框架，引入 API 调用序列、网络行为等动态特征以提升对抗混淆和加壳的能力。通过微调轻量级开源模型替代商用 API，构建本地化推理引擎，显著降低成本并提高响应效率。扩展模型对勒索软件、木马等多类恶意软件的检测验证，同时增强提示注入防护机制，结合可解释性增强技术，使分析过程更透明可靠。最终形成适应高对抗、低成本需求的下一代智能恶意软件检测系统。

（五）对构建智能体“分析师”的逐步探索总结

论文	LLM for code analysis	Beyond Classification	DeGPT	AV-Agent
对构建 agent 的探索	评判 LLM 的代码理解与分析能力	评判 LLM 在 Android 恶意软件审计中的能力	弱自治智能体系统框架	类智能体框架
输入	非混淆或经过混淆的代码文件	可达函数抽象成的四元组<signature, summary, sensitivity score>	反编译器的输出	原始二进制先经特征提取与分类器得到的置信分数、关键字符/特征
输出	对代码功能的自然语言解释	敏感函数排序、证据→函数归因、行为报告综合、对误判样本的反驳/判别	可读性更强的反编译 C 代码	恶意软件分类结论 + 结构化分析过程/解释
LLM 幻觉问题的处理	人工核验把错误解释剔除，用多指标检验输出一致性	用“证据真实性/引用有效性”避免 LLM 乱挂证据、编造引用	Operator + MSSC 过滤并报告能检测出的有害修改	两阶段与结构化 prompt 降低发散；强调生成不确定性，做稳定性测试
适用场	代码理解能	恶意行为审计评测 /	逆向工程 /	可解释恶意软件

景	力评测 / 混淆代码分析	证据链分析	反编译可读性优化	分类
---	--------------	-------	----------	----

综合来看，这四篇文章对构建智能体“分析师”进行了不同阶段的探索。**LLM for code analysis** 主要探讨了大语言模型（LLMs）在代码分析中的有效性，尤其是它们在自动化恶意软件分析中的应用。文章评估了 LLMs 在代码审计中的局限性，强调了模型生成的解释性和可追溯性的问题。**Beyond Classification** 提出了“MalEval”评估框架，专注于如何使用 LLM 支持恶意软件行为审核。与第一篇文章侧重于代码分析不同，这篇文章深入探讨了 LLMs 在恶意软件行为审计中的应用，尤其是对于行为总结、证据归因以及行为合成的能力评估。通过设计多层次的任务和分析指标，文章展示了 LLMs 在帮助分析师提供行为报告和过滤误报时的潜力与不足。

DeGPT 关注于逆向工程领域，提出了 **DeGPT** 框架，旨在优化反汇编器输出，提升反向工程师对恶意软件的理解。文章展示了如何通过 LLM 来优化反汇编代码的可读性，包括变量重命名、结构简化和添加注释。通过提出三角色机制和微片段语义计算（MSSC），构建出了一个弱自治智能体系统，是多智能体协作解决实际问题的雏形。**AV-agent** 结合了分层特征提取和两阶段推理。与其他方法不同，**AV-Agent** 通过融合机器学习和 LLM 的推理能力，提供了一个可解释的恶意软件分析系统。通过实验展示了 **AV-Agent** 在恶意软件分类中的卓越性能，并强调了多层特征提取和推理机制在提高分类准确性和透明度方面的重要作用，其中，采用独立 ML 模型作为感知模块，LLM 作为工具，深刻体现了智能体的感知、工具调用与自我反馈回路，是对构建智能体“分析师”这一任务较为成熟、有前瞻性的探索。

总的来说，这四篇文章虽然都涉及到使用大语言模型（LLMs）在恶意软件分析中的应用，但它们的关注点和方法各有不同。第一篇强调了 LLM 在代码分析中的潜力与局限，第二篇则深入探讨了 LLM 在行为审计中的应用，并提出了一个多任务的评估框架。第三篇则专注于反向工程中的优化问题，提出了一个弱自治智能体系统框架 **DeGPT**，初步探索了多智能体协作解决恶意代码分析问题的可能性。而第四篇则提出了一个类智能体框架 **AV-Agent**，结合了机器学习作为感知模块和 LLM 的推理能力，具有较强的可解释性和实际应用价值。总体来

看，四篇文章从分析 LLM 在恶意代码分析方面的局限性到探索构建智能体“分析师”的可能性，为之后真正构建起能够主动规划、决策和执行整个分析流程的智能体“分析师”打下了坚实的基础。

五、总结与展望

本次综述回顾了恶意软件分析技术从特征工程到智能体构建的四个主要演进阶段，每个阶段都在解决上一阶段的局限性并引入新的核心技术：

（一）研究演进总结

恶意软件分析技术的演进历程可以划分为四个显著阶段，这一过程体现了从手工特征工程向自主智能分析的逐步跨越。在 Level 1“传统恶意软件检测技术”阶段，研究的核心在于如何通过程序分析技术将原始二进制代码转化为可区分的特征空间，主要 workflow 侧重于提取语义、关系、时序及结构特征来构建中间表示。例如，MaMaDroid 通过马尔可夫链对 API 调用的时序逻辑进行建模，以捕捉程序运行过程中的行为逻辑；HinDroid 利用异构信息网络和元路径提取来捕捉深层结构化关系，从而提升抗逃避能力；而为了应对模型老化问题，研究者引入了基于 API 文档知识图谱的语义聚类方法，通过特征替换延缓性能衰减。这一阶段虽然为分类奠定了基础，但其局限性在于高度依赖人工设计的特征提取与建模。

随着攻防对抗的升级，Level 2“表征驱动与鲁棒化分类”阶段致力于解决现实世界中数据分布变化带来的严峻挑战，核心任务转向解决标签噪声、概念漂移及样本不均衡问题。为了提升模型的泛化能力与鲁棒性，研究者提出了差分训练框架，通过双模型训练与离群值检测来识别并修正噪声标签；利用分层对比学习和伪损失采样技术实现持续学习，以应对恶意软件演化导致的概念漂移；并通过对抗域自适应技术对齐漂移前后的数据分布，使模型能够学习不变特征。此外，该阶段还深入探讨了模型的安全性，研究了针对黑盒模型的功能保持型对抗攻击（如利用代码空洞的 Code-Cave 攻击）以及后门净化防御（如 PBP 框架），旨在构建在对抗环境下依然稳健的分类模型。

突破了单纯的分类任务限制，Level 3“走向‘理解’恶意软件”阶段利用大语言

模型（LLM）强大的代码理解与生成能力，致力于实现深层语义恢复与行为解释。在这一阶段，研究重点转向了利用生成式模型解决传统判别式模型无法处理的任务。例如，ReSym 利用 LLM 从剥离符号的二进制文件中恢复变量名与数据结构，解决了逆向工程中的语义缺失难题；ForeDroid 通过场景感知建模，结合敏感 API 调用链为恶意行为提供细粒度的解释。同时，MalGuard 和 SocketAI 等工作利用提示词工程与思维链技术，有效提升了恶意包检测的准确性与可解释性。尽管取得了显著进展，该阶段仍面临幻觉问题、上下文长度限制及推理成本高昂等挑战，限制了其在大规模场景下的直接应用。

目前的最前沿探索集中在 Level 4“构建智能体‘分析师’”阶段，旨在打造能够像人类专家一样主动规划、决策并执行完整分析流程的自主智能体。这一阶段不再局限于单一任务的辅助，而是强调系统的自主性与闭环反馈。例如，DeGPT 设计了“裁判-顾问-操作员”的多角色协作机制来优化反编译代码，通过角色间的交互提升输出质量；AV-Agent 构建了“感知-推理-决策”的自修正闭环，通过两阶段推理和显式展开分析过程来提升决策的稳定性与逻辑性。为了验证这些能力的有效性，研究者还建立了如 MalEval 等系统性的代码分析能力评测基准，重点评估 LLM 在细粒度行为审计中的表现。总体而言，这一演进路径清晰地展示了恶意软件分析系统正朝着自动化、可解释、强适应且低成本的下一代分析平台迈进。

（二）未来展望与挑战

恶意软件分析的终极目标是构建一个自动化、可解释、强适应且低成本的下一代分析系统。为了实现这一愿景，未来的研究需要在以下几个维度寻求突破：

1. 从“工具调用”向“认知智能”跨越

目前的 Level 4 智能体虽然能调用工具，但在复杂逻辑推理和未知威胁判定上仍有不足。未来需要增强智能体的自主规划能力，使其不仅是执行特定子任务（如反编译）的工具，而是具备决策能力的“分析师”。

2. 可信性与鲁棒性的提升

随着 LLM 在安全领域的深入应用，如何解决“幻觉”导致的错误归因（如 ForeDroid 中提到的证据归因问题）是关键。此外，针对智能体本身的对抗性攻击（如提示词注入）将成为新的攻防焦点，必须提升系统在对抗环境下的鲁棒性。

3. 解决“实验与现实”的偏差

正如 TESSERACT 研究指出的，实验室环境下的高准确率往往带有时间与空间偏差。未来的研究应更注重在真实时间轴上的抗老化能力测试（AUT 指标），以及在样本极端不平衡的真实场景下的有效性验证。

4. 效率与成本的平衡

LLM 的高推理成本限制了其大规模实时应用。未来的方向可能包括研发针对特定安全任务微调的轻量级模型，或者采用大小模型协同（如 MalGuard 中的离线特征构建+轻量级 ML 检测）的混合架构。

论文阅读列表

盖乐：

MAMADROID: Detecting Android Malware by Building Markov Chains of Behavioral Models

HinDroid: An Intelligent Android Malware Detection System Based on Structured Heterogeneous Information Network

Slowing Down the Aging of Learning-Based Malware Detectors With API Knowledge

DawnGNN: Documentation augmented windows malware detection using graph neural network

陈昕怡：

Decoding the Secrets of ML in Malware Classification

Differential Training: Reducing Label Noise for Android Malware Detection

Continuous Learning for Android Malware Detection

Revisiting Concept Drift in Windows Malware Detection: Adaptation with Graph Neural Networks

王迪：

TESSERACT: Eliminating Experimental Bias in Malware Classification across Space

and Time

Functionality-preserving black-box optimization of adversarial windows malware

Make your IoT environments robust against adversarial machine learning malware threats: a code-cave approach

PBP: Post-training Backdoor Purification for Malware Classifiers

贾泽栋:

ReSym: Harnessing LLMs to Recover Variable and Data Structure Symbols from Stripped Binaries

MalGuard: Towards Real-Time, Accurate, and Actionable Detection of Malicious Packages in PyPI Ecosystem

Leveraging Large Language Models to Detect npm Malicious Packages

ForeDroid: Scenario-Aware Analysis for Android Malware Detection and Explanation

陈奕昊:

Large Language Models for Code Analysis: Do LLMs Really Do Their Job?

Beyond Classification: Evaluating LLMs for Fine-Grained Automatic Malware Behavior Auditing

DeGPT: Optimizing Decompiler Output with LLM

AV-Agent: A Bottom-Up Interpretable Malware Classifier Based on Large Language Models.